在数据集选择上，我采用的是一个分阶段的策略，而不是一开始就直接使用大规模复杂数据。主要原因是，我当前的项目已经完成了核心 pipeline 的搭建，包括 inference、diagnostic analysis、calibration、reranking 以及 robustness 分析，因此接下来的重点不再是“能不能跑通”，而是“能不能在更真实的数据上稳定复现并得到具有统计意义的结论”。

具体来说，在第一阶段，我选择使用 MovieLens 的稳定版本作为基础数据集。这样做的目的，是先在一个结构规范、社区广泛使用的 benchmark 上验证整个 uncertainty-aware recommendation pipeline 的可运行性和可复现性。MovieLens 的数据结构非常清晰，用户—物品交互关系明确，适合我去验证 calibration、confidence-correctness 关系以及 reranking 等基础实验，同时也方便快速定位工程问题和调试流程。

在第二阶段，我会引入 Amazon Reviews’23 作为主要实验数据集。相比 MovieLens，这个数据集包含更加丰富的文本信息，例如用户评论、商品描述以及类别信息，这对于 LLM 推荐任务来说非常关键。因为我的方法本质上依赖模型对文本语义的理解来进行推荐判断和不确定性估计，所以在这个数据集上可以更真实地评估 uncertainty 的行为，包括 calibration 是否有效、是否存在 popularity bias，以及在 noisy 或扰动条件下系统的稳定性表现。

在实现层面，我不会直接把新数据硬接到现有代码里，而是会新增一套统一的数据处理模块。具体来说，我会先实现数据下载与原始数据读取模块，将 MovieLens 和 Amazon Reviews’23 的原始数据分别解析为统一的内部格式；然后实现一个通用的预处理与样本构造流程，把原始交互数据转换为 pointwise recommendation 所需的样本结构。每一条样本都会包含用户历史、候选物品的文本描述、标签以及对应的 popularity group，这样可以无缝接入我现有的 inference 和后续分析 pipeline。

此外，我还会补充数据集配置文件，将不同数据集的路径、过滤条件、候选采样策略等参数统一放在配置层中，而不是写死在代码里。这样一来，我后续在切换数据集、调整样本规模或修改实验设置时，只需要修改配置文件即可，不需要频繁改动代码逻辑。

在数据准备完成之后，我会按照现有 pipeline 的流程，依次运行 inference、evaluation、calibration 和 reranking，在真实数据上重新跑一整套实验。同时，我也会在 Amazon 数据上重点开展更复杂的实验，例如多源 uncertainty 建模、不同 λ 的敏感性分析，以及更强噪声条件下的 robustness 实验，从而使整个工作从原型验证进一步提升到具有论文投稿水平的实验体系。

总体来说，这一数据层的改造并不是简单地“换一个数据集”，而是将项目从小规模验证环境，系统性地迁移到更大规模、更真实、更适合 LLM 推荐任务的实验场景中，使后续所有关于 uncertainty 的分析、建模和决策方法都能够在更有说服力的数据基础上得到验证。

本项目采用分阶段数据集策略。第一阶段选用 MovieLens 稳定版本作为基准数据集，用于验证 uncertainty-aware recommendation pipeline 在真实推荐数据上的可运行性、可复现性和基础现象有效性。MovieLens 官方当前将 MovieLens 32M 列为“recommended for new research”，并明确将其描述为 stable benchmark dataset；同时，MovieLens 25M 也是稳定版本，包含评分、标签以及 tag genome 等辅助信息，适合作为经典推荐 benchmark。第二阶段选用 Amazon Reviews'23 作为主实验数据源，因为该数据集由 McAuley Lab 于 2023 年收集，包含用户评论文本、评分、商品 metadata 与 user-item / bought-together links，更适合 LLM 在文本语义条件下进行推荐判断与 uncertainty 建模。该数据集官方还提供 native json 与 Hugging Face datasets 两种加载方式，以及 0-core/5-core 处理版本，便于根据实验规模进行裁剪。

选择 MovieLens 的原因在于其结构规范、社区使用广泛、适合快速验证 calibration、reranking 与 bias 分析；选择 Amazon Reviews'23 的原因在于其文本信息更加丰富、长尾属性更明显、与 LLM recommendation 的任务设定更贴近，更适合作为 uncertainty-aware decision making 与 robustness analysis 的主要实验场景。当前项目总览里也已经明确指出，后续应当迁移到更大规模、更真实的数据集上，以增强统计稳定性，并进一步扩展多源 uncertainty 融合与更复杂 robustness setting。

具体操作顺序。先不要删你现有的小样本数据，它仍然是最重要的 sanity check。然后新增一个真实数据处理分支，先接 MovieLens 1M 或 25M。1M 更适合快速改代码和调试，25M 更适合做正式 baseline。跑通以后，再接 Amazon Reviews'23 的一个中等类目子集，比如 Movies/TV、Books、Digital Music 或 Beauty 这类文本和语义相对清晰的 domain。这样可以先在 MovieLens 上稳定完成 calibration、reranking、basic robustness，再在 Amazon 上真正做 uncertainty、文本噪声和长尾偏差研究。这个推进路径和你原来的实现方案也是一致的，也就是先保证闭环与分析，再进入更强的数据与更复杂的方法。

第一步，需要新增数据下载与预处理入口。建议在项目里增加一个 `scripts/download_datasets.py`，专门负责下载或检查原始数据文件是否存在；同时增加 `src/data/raw_loaders.py`，把不同数据源的读取逻辑隔离出来。这个文件里至少要有两个 loader，一个是 `load_movielens(...)`，一个是 `load_amazon_reviews23(...)`。MovieLens 的 loader 主要读取 ratings、movies、tags、links 这些表，再拼出 item 文本；Amazon 的 loader 则读取 reviews、metadata，按 asin 或 parent_asin 对齐，抽取 title、description、category、price 等字段，构造成统一 item schema。这样做的好处是后面的 sample builder、prompt builder、candidate sampler 都不用关心原始数据到底来自哪个数据集。

第二步，需要把现有 `src/data/preprocess.py` 升级成“多数据集预处理入口”，而不是只服务当前小样本。最好的方式是把它改造成这样一种结构：输入数据集名、原始路径、输出路径、最小交互数、候选采样数、文本字段策略，输出统一格式的 `train.jsonl`、`valid.jsonl`、`test.jsonl`，以及 `items.jsonl`、`users.jsonl`、`popularity_stats.json`。这里最关键的是统一样本格式，因为现有 pointwise pipeline 已经证明这种格式是有效的：每条样本要至少包含 `user_id`、`history`、`candidate_item_id`、`candidate_title`、`candidate_text`、`label`、`target_popularity_group`。在第一阶段总结里已经明确说明，这种结构化 prediction 流是整个项目能成立的基础。

第三步，需要新增一个真正的数据集配置层。建议在 `configs/data/` 下新建至少四个文件：`movielens_1m.yaml`、`movielens_25m.yaml`、`amazon_reviews23_beauty.yaml`、`amazon_reviews23_movies.yaml`。每个配置文件里写清楚数据路径、交互过滤阈值、历史长度、候选集大小、文本字段来源、是否使用评论文本、是否启用 5-core。之前的项目结构设计本来就预留了 `configs/data/`、`configs/model/`、`configs/exp/` 这套骨架，现在正好把它真正落地。

第四步，需要新增一个“统一样本构造器”。建议增加 `src/data/sample_builder.py`。这个文件负责把原始交互表变成pointwise LLM 推荐所需的样本。它至少要做四件事。第一，根据时间顺序给每个用户切分历史和未来交互。第二，从未交互 item 中采负样本，形成 candidate set。第三，把 item metadata 拼成 prompt 所需文本，比如 title + genre 或 title + description。第四，基于 item 交互次数计算 popularity，并把 item 分到 head、mid、tail。后面的 popularity bias、exposure analysis、robustness 分析，全都依赖这一步统一输出。原先的诊断分析模块里也明确把 confidence、popularity、exposure 的关系作为关键研究问题，所以这一层必须标准化。

第五步，需要把候选采样逻辑单独强化。现在已经有 `src/data/candidate_sampling.py` 的雏形，但迁移到真实数据后，这个文件必须支持更多策略。最低限度要有 random negative sampling，最好再补一个 popularity-aware sampling，后面有余力再加 semantic-hard negative sampling。因为如果全是太容易区分的负样本，confidence 结果会显得过于理想，calibration 和 uncertainty 的研究价值会被削弱。这个点虽然之前的小样本能跑，但一上真实数据就会成为决定实验质量的关键。

第六步，需要补 item 文本生成模块。建议新增 `src/data/text_builder.py`。MovieLens 本身文本不强，所以可以先把 `title + genres + tags` 拼成基础描述；Amazon Reviews'23 则可以用 `title + description + categories (+ 可选 summary)` 构建候选 item 文本。这个模块的作用，是把不同数据集的异构字段转换成你 prompt 能稳定消费的统一自然语言格式。否则后面的 prompt_builder 会被不同数据集绑死，很难维护。

第七步，需要增加 dataset registry。建议新增 `src/utils/registry.py` 或直接在其中增加 `DATASET_REGISTRY`，把 `"movielens_1m" -> loader + builder config`、`"amazon_beauty_2023" -> loader + builder config` 这种映射写进去。这样主程序只要传 `--data_config configs/data/amazon_reviews23_beauty.yaml`，系统就能自动知道该走哪条 preprocessing 路线，而不用在主程序里写一堆 if-else。这类工程化动作和你现在整个 pipeline 自动化方向是一致的。

第八步，需要新增两个数据处理主入口。一个是 `main_preprocess.py`，负责把原始数据变成统一 processed data；另一个是 `main_build_samples.py`，负责从 processed data 生成 pointwise 的 train/valid/test 样本。如果想入口少一点，也可以把两步合并成一个 `main_prepare_data.py`。但我更推荐拆开，因为后面会经常需要反复改候选采样和样本构造，而不一定每次都要重跑原始清洗。

第九步，需要补下载和目录。建议项目里新增如下目录：`data/raw/movielens_1m/`、`data/raw/movielens_25m/`、`data/raw/amazon_reviews23/`、

`data/processed/movielens_1m/`、`data/processed/movielens_25m/`、`data/processed/amazon_reviews23_beauty/`。此外，在 `scripts/` 下加 `download_movielens.sh`、`download_amazon_reviews23.sh`、`prepare_movielens.sh`、`prepare_amazon_beauty.sh`。项目现在已经从“写几个脚本跑一下”进入“需要长期复现实验”的阶段，所以数据准备也必须脚本化。我自己关于 pipeline 规范化的总结里，其实已经反复强调了这一点。

第十步，需要改 README 或 proposal 中的数据部分。正式写法可以分成四段。第一段写数据选择原则，也就是交互历史、文本丰富度、长尾结构、可转成 pointwise 样本。第二段写两个阶段的数据路线。第三段写每个数据集的下载来源与预处理方式。第四段写最终统一样本格式和目录结构。你之前已经有一份项目总览和实现方案，只需要把“数据集选择与获取方案”这一节单独加进去即可。

实际命令行操作步骤。先从官方页面下载 MovieLens 稳定版本压缩包，放到 `data/raw/movielens_25m/` 或 `data/raw/movielens_1m/` 下；MovieLens 官方目前明确给出了 32M、25M 等稳定 benchmark 入口。然后从 Amazon Reviews'23 官方页面进入 Hugging Face datasets 或 GitHub 页面，选一个类目子集先做，比如 Beauty 或 Movies/TV；官方文档明确支持 native json 和 Hugging Face datasets 两种方式，也提供了 0-core/5-core 处理版本说明。本地拿到原始文件后，第一轮先跑 `main_preprocess.py --config configs/data/movielens_1m.yaml`，再跑 `main_build_samples.py --config configs/data/movielens_1m.yaml`，检查是否成功生成 `train.jsonl / valid.jsonl / test.jsonl`。确认样本格式正确以后，再直接复用现有的 `main_infer.py -> main_eval.py -> main_calibrate.py -> main_rerank.py` 整条链路去跑真实数据实验。由于已经把输出目录按 exp_name 规范化了，这一步工程上其实是顺的。

写哪些代码以及必须新增或重写的，是 `main_preprocess.py`、`main_build_samples.py`、`src/data/raw_loaders.py`、`src/data/sample_builder.py`、`src/data/text_builder.py`，以及 `configs/data/` 下的数据集 yaml。必须修改的，是 `src/data/preprocess.py`、`src/data/dataset.py`、`src/data/candidate_sampling.py`、`src/data/popularity.py`、`README.md`。建议新增但不是绝对必须的，是 `scripts/download_*.sh`、`scripts/prepare_*.sh`、`src/utils/registry.py`。而像 `src/eval/calibration_metrics.py`、`src/analysis/popularity_bias.py`、`src/eval/ranking_metrics.py`、`src/eval/bias_metrics.py` 这些模块，原则上大多不用大改，因为它们关心的是统一 dataframe 或 prediction schema，而不是底层原始数据来自哪里。这一点和之前对 pipeline 设计问题的判断是完全一致的。

proposal

本项目采用分阶段数据集策略，以兼顾工程可控性与研究有效性。第一阶段选用 MovieLens 稳定 benchmark 数据集作为基础实验平台，用于验证 uncertainty-aware recommendation pipeline 在真实推荐数据上的可运行性、可复现性及基础现象有效性。第二阶段选用 Amazon Reviews'23 数

据集作为主实验场景，利用其丰富的评论文本、商品元数据与用户—物品交互信息，系统研究 LLM recommendation 中的 confidence calibration、popularity bias、exposure shift 及 robustness 问题。在实现上，项目将新增统一的数据下载、预处理与样本构造模块，将不同来源的数据映射到一致的 pointwise recommendation 样本格式，包含用户历史、候选物品文本、标签与 popularity group，从而无缝接入现有的 inference、evaluation、calibration 与 reranking pipeline。通过这一数据层改造，项目将从小规模原型验证进一步推进到更大规模、更多文本语义、更接近真实场景的实验体系。